PRÁCTICA 4 de JAVA: bases de datos

9/11/2026

Fecha entrega	16 noviembre	**Fecha defensa**	16 noviembre
Tipo	Individual	**Entrega**	Github classroom

Desarrolla una aplicación grafica con Java para la gestión de una empresa y su base de datos, según el MVC. Crea un paquete por capa (modelo, vista, controlador).

Requisitos:

1. Base de datos (libre, distinta al ejemplo, con el visto bueno del profesor)

1.1. Dos tablas con relación 1→N del tipo A → B (P. Ej., cliente → cuenta).
1.2. Tipos obligatorios a usar entre las dos tablas: int, float, String y LocalDate (columna DATE).
1.3. La Tabla A (cliente) servirá para validación y al menos tendrá los campos: usuario, contraseña y una imagen (en la BD (BLOB) o en disco (ruta y nombre del fichero), a tu elección.
1.4. PK y FK con el formato cliNumero (3 primeras letras de la tabla donde es PK + nombre del campo).
1.5. Campo calculado en la tabla A. Puede ser suma, media, máximo o mínimo de un campo de la tabla B. P. Ej. cliente.saldoTotal sería suma de todas los cuenta.saldo de ese cliente.
1.6. Entrega el diagrama E/R y el script SQL.
1.7. La BD se ejecuta en un contenedor Docker (MariaDB, MySQL o PostgreSQL). Entrega los comandos necesarios para su puesta en marcha.

2. Vista: un JFrame con varios JPanel. Partes:

2.1. Barra de menú y menú con las opciones : **Validar** (entrar/salir), **Visualizar** (detalle/resumen) y **Acerca de**, con opciones activadas o desactivadas según el estado de la aplicación.
2.2. **Validar/entrar**: pedirá usuario y contraseña. Serán consultados con PreparedStatement. Si es correcto, se activa el resto del menú. La conexión con el administrador es automática y transparente.
2.3. **Detalle**: muestra una a una las cuentas del cliente validado.
- Modificar la fecha con DatePicker.
- Botones de primero, anterior, siguiente y último, directamente sobre el ResultSet.
- Botón Guardar, que actualiza solo la cuenta visualizada.
- Modificar también el campo calculado (cuenta.saldo).

2.4. **Resumen**: muestra todos los datos de la tabla A, incluido el campo calculado y la imagen. Además habrá un JTable con los campos de la tabla B.
- Botón "Calcular": muestra el resultado en un JOptionPane.INFORMATION_MESSAG y actualiza el campo calculado en la BD.

2.5. **Acerca de**: JDialog modal personalizado (autores, fecha, versión).
2.6. Mensajes al usuario con JOptionPane.WARNING_MESSAGE (Advertencia) o JOptionPane.ERROR_MESSAGE (Error).

3. Modelo: una clase POJO por tabla, solo con atributos, constructores y get/set. Sin código sobre la BD.

4. Controlador (núcleo de la práctica): cuatro clases.

4.1. ConexionBD: abre, cierra y gestiona la conexión (la única que puede ser estática) y proporciona los Statement para detalle y resumen.
- (Voluntario) Lee los parámetros de un .properties.

4.2. CuentaDetalleDAO: Statement sensible y actualizable. Con métodos para mover el resultSet o navegar (inicio, fin, siguiente, anterior), guardar y modificar, y además devolver el objeto actual (sobre el que está el resultSet) de la tabla B (Cuenta).
4.3. CuentaDAO: Statement simple. Devuelve una Collection<Cuenta> para el resumen.
4.4. ClienteDAO: PrepareStatement simple. Valida el login y lee y actualiza objetos Cliente.
4.5. Los errores se gestionan con una excepción propia (DAOException).
4.6. Los recursos (Connection, Statement, ResultSet) se cierran con try-with-resources. El DAO del panel Detalle implementa AutoCloseable y se cierra al salir o al cambiar de panel.

5. Voluntarias (sube nota):

- Contraseñas con hash.
- Alta y baja de tabla B (CRUD completo).
- Filtro/búsqueda en el resumen.
- Exportar el resumen a CSV o PDF.
- Ramos por paso o funcionalidad en git.

Ayuda: Guía de uso de PostgreSQL en Docker.

7. Guía de Git para el alumnado

El repositorio forma parte de la entrega: su historial muestra el trabajo Paso a Paso y la autoría. Se valora dentro de "calidad y buenas prácticas" y "documentación".

Qué se sube y qué no

- **Sí**: código fuente, script .sql, docker-compose.yml, carpeta lib/ con los .jar (si no, el proyecto Ant no compila en otro equipo), diagrama E/R (imagen y fichero de origen) y README.md.
- **No**: contraseñas reales, db.properties con datos de conexión, carpetas generadas (build/, dist/), configuración privada de NetBeans y datos del volumen de Docker.
- En lugar de db.properties, se sube un db.properties.example con valores de ejemplo.

.gitignore (punto de partida para NetBeans con Ant)

```
/build/
/dist/
/nbproject/private/
*.class
*.log
db.properties
.env
/data/
```

Crea el .gitignore antes del primer commit. Si un fichero ya se subió por error, ignorarlo después no basta: hay que quitarlo del repositorio con git rm --cached.

Commits y flujo de trabajo

- Commits pequeños y frecuentes, con mensajes que digan qué se hizo: "Añade CuentaDetalleDAO con navegación", no "cambios" ni "arreglado".
- Rama main siempre funcional. Las ramas (una por Paso o por funcionalidad) son opcionales.
- Si trabajan en pareja: git pull antes de empezar y git push al terminar cada bloque de trabajo, para evitar conflictos.

README.md bien completado

Es lo primero que lee quien abre el repositorio. Contenido mínimo:

1. **Título, autores y curso.**
2. **Descripción** de la aplicación en 3-4 líneas.
3. **Esquema E/R** insertado como imagen (docs/er.png) y una tabla con las tablas y campos, indicando tipos, PK y FK.
4. **Tecnologías y versiones**: NetBeans 31, JDK, motor de BD elegido, Docker y librerías de lib/.
5. **Cómo ejecutarla**, paso a paso, de modo que otra persona pueda reproducirlo.
6. **Usuarios de prueba** (usuario y contraseña de ejemplo).
7. **Estructura de paquetes** (modelo, vista, controlador) con una línea por clase principal.
8. **Capturas** de login, detalle, resumen y Acerca de.
9. **Decisiones de diseño**, por ejemplo cómo se guarda la imagen y por qué.
10. **Estado de los Pasos** como lista de tareas y apartados voluntarios realizados.

Plantilla de ejmplo:

```
# Nombre de la aplicación

Autores: Nombre Apellido, Nombre Apellido · Curso 20XX/XX

## Descripción
Qué hace la aplicación en 3-4 líneas.

## Esquema E/R
![Esquema E/R](docs/er.png)

| Tabla | Campo | Tipo | Clave |
|---|---|---|---|
| cliente | cliNumero | int | PK |
| cuenta | cueNumero | int | PK |
| cuenta | cliNumero | int | FK → cliente |

## Tecnologías
NetBeans 31 · JDK XX · MariaDB XX (Docker) · LGoodDatePicker X.X

## Cómo ejecutarla
1. Arrancar la BD: `docker ….`
2. Copiar `db.properties.example` a `db.properties` y revisar los valores.
3. Abrir el proyecto en NetBeans y comprobar que `lib/` está en las bibliotecas.
4. Ejecutar la clase principal `vista.Principal`.

## Usuarios de prueba
| Usuario | Contraseña |
|---|---|
| ana | 1234 |

## Estructura
- `modelo/`: clases Cliente y Cuenta
- `vista/`: JFrame y paneles
- `controlador/`: ConexionBD y DAO

## Capturas
![Login](docs/login.png)
```

Decisiones de diseño
Por qué la imagen va en la BD (o en disco).

Estado
- [x] Paso 1 – BD y Docker
- [ ] Paso 2 – Conexión y DAO
- [ ] Voluntarias: loggin